



COMP 412
FALL 2012

Top-down Parsing Recursive Descent & LL(1)

Comp 412

The lecture is self consistent, but the order of the three productions for Factor in the expression grammar is different than in 2e

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.
Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.
Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.



Roadmap (Where are we?)

We set out to study parsing

- Specifying syntax
 - Context-free grammars ✓
- Top-down parsers
 - Algorithm & its problem with left recursion ✓
 - Ambiguity ✓
 - Left-recursion removal ✓
- Predictive top-down parsing
 - The LL(1) condition ✓
 - Simple recursive descent parsers today
 - First and Follow sets today
 - Table-driven LL(1) parsers today

Predictive Parsing



Basic idea

Given $A \rightarrow \alpha \mid \beta$, the parser should be able to choose between α & β

FIRST sets

For some *rhs* $\alpha \in G$, define **FIRST(α)** as the set of tokens that appear as the first symbol in some string that derives from α

That is, $\underline{x} \in \text{FIRST}(\alpha)$ iff $\alpha \Rightarrow^* \underline{x}\gamma$, for some γ

The LL(1) Property

If $A \rightarrow \alpha$ and $A \rightarrow \beta$ both appear in the grammar, we would like

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$$

This would allow the parser to make a correct choice with a lookahead of exactly one symbol !

This is almost correct
See the next slide



Predictive Parsing

What about ε -productions?

⇒ They complicate the definition of $LL(1)$

If $A \rightarrow \alpha$ and $A \rightarrow \beta$ and $\varepsilon \in \text{FIRST}(\alpha)$, then we need to ensure that $\text{FIRST}(\beta)$ is disjoint from $\text{FOLLOW}(A)$, too, where

$\text{FOLLOW}(A)$ = the set of terminal symbols that can immediately follow A in a sentential form

Define $\text{FIRST}^+(A \rightarrow \alpha)$ as

- $\text{FIRST}(\alpha) \cup \text{FOLLOW}(A)$, if $\varepsilon \in \text{FIRST}(\alpha)$
- $\text{FIRST}(\alpha)$, otherwise

Then, a grammar is $LL(1)$ iff $A \rightarrow \alpha$ and $A \rightarrow \beta$ implies

$$\text{FIRST}^+(A \rightarrow \alpha) \cap \text{FIRST}^+(A \rightarrow \beta) = \emptyset$$

The intuition is straightforward. If A expands with 2 right hand sides, those rhs' must have distinct first symbols.

We only need FIRST^+ sets because of ε -productions.



Predictive Parsing

Given a grammar that has the $LL(1)$ property

- Can write a simple routine to recognize each *lhs*
- Code is both simple & fast

Consider $A \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$, with

$$\text{FIRST}^+(A \rightarrow \beta_i) \cap \text{FIRST}^+(A \rightarrow \beta_j) = \emptyset \text{ if } i \neq j$$

```
/* find an A */  
if (current_word  $\in$  FIRST( $A \rightarrow \beta_1$ ))  
    find a  $\beta_1$  and return true  
else if (current_word  $\in$  FIRST( $A \rightarrow \beta_2$ ))  
    find a  $\beta_2$  and return true  
else if (current_word  $\in$  FIRST( $A \rightarrow \beta_3$ ))  
    find a  $\beta_3$  and return true  
else  
    report an error and return false
```

Grammars with the $LL(1)$ property are called predictive grammars because the parser can “predict” the correct expansion at each point in the parse.

Parsers that capitalize on the $LL(1)$ property are called predictive parsers.

One kind of predictive parser is the recursive descent parser.

Of course, there is more detail to “find a β_i ” (p. 108-111 in EaC2e)



Recursive Descent Parsing

Recall the expression grammar, after transformation

0	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
1	<i>Expr</i>	$\rightarrow$	<i>Term Expr'</i>
2	<i>Expr'</i>	$\rightarrow$	$+ \textit{Term Expr'}$
3		$ $	$- \textit{Term Expr'}$
4		$ $	ϵ
5	<i>Term</i>	$\rightarrow$	<i>Factor Term'</i>
6	<i>Term'</i>	$\rightarrow$	$* \textit{Factor Term'}$
7		$ $	$/ \textit{Factor Term'}$
8		$ $	ϵ
9	<i>Factor</i>	$\rightarrow$	$(\textit{Expr})$
10		$ $	<u>number</u>
11		$ $	<u>id</u>

This produces a parser with six mutually recursive routines:

- *Goal*
- *Expr*
- *EPrime*
- *Term*
- *TPrime*
- *Factor*

Each recognizes one *NT* or *T*

The term descent refers to the direction in which the parse tree is built.

Recursive Descent Parsing (Procedural)



A couple of routines from the expression parser

Goal()

```
token ← next_token();  
if (Expr() = true & token = EOF)  
    then next compilation step;  
else  
    report syntax error;  
    return false;
```

Expr()

```
if (Term() = false)  
    then return false;  
else return Eprime();
```

looking for Number, Identifier, or
"(", found token instead, or failed
to find Expr or ")" after "("

Factor()

```
if (token = Number) then  
    token ← next_token();  
    return true;  
else if (token = Identifier) then  
    token ← next_token();  
    return true;  
else if (token = Lparen)  
    token ← next_token();  
    if (Expr() = true & token = Rparen) then  
        token ← next_token();  
        return true;  
    // fall out of if statement  
    report syntax error;  
    return false;
```

EPrime, Term, & TPrime follow the same basic lines (Figure 3.10, page 111, EaC2e)



What If My Grammar Is Not LL(1) ?

Can we transform a non-LL(1) grammar into an LL(1) grammar?

- In general, the answer is no
- In some cases, however, the answer is yes

Assume a grammar G with productions $A \rightarrow \alpha \beta_1$ and $A \rightarrow \alpha \beta_2$

- If α derives anything other than ε , then

$$\text{FIRST}^+(A \rightarrow \alpha \beta_1) \cap \text{FIRST}^+(A \rightarrow \alpha \beta_2) \neq \emptyset$$

- And the grammar is not LL(1)

If we pull the common prefix, α , into a separate production, we may make the grammar LL(1).

$$A \rightarrow \alpha A', A' \rightarrow \beta_1 \text{ and } A' \rightarrow \beta_2$$

Now, if $\text{FIRST}^+(A' \rightarrow \beta_1) \cap \text{FIRST}^+(A' \rightarrow \beta_2) = \emptyset$, G may be LL(1)



What If My Grammar Is Not LL(1) ?

Left Factoring

*For each NT A
find the longest prefix α common to 2 or more alternatives for A
if $\alpha \neq \epsilon$ then
replace all of the productions
 $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \alpha \beta_3 \mid \dots \mid \alpha \beta_n \mid \gamma$
with
 $A \rightarrow \alpha A' \mid \gamma$
 $A' \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n$
Repeat until no NT has alternative rhs' with a common prefix*

This transformation makes some grammars into LL(1) grammars
There are languages for which no LL(1) grammar exists



Left Factoring Example

Consider a short grammar for subscripted identifiers

0	<i>Factor</i>	→	<u>name</u>
1			<u>name</u> [<i>ArgList</i>]
2			<u>name</u> (<i>ArgList</i>)
3	<i>ArgList</i>		<i>Expr</i> <i>MoreArgs</i>
4	<i>MoreArgs</i>	→	ϵ <i>Expr</i> <i>MoreArgs</i>
5			ϵ

Common prefix is name.
Introduce a new production
Factor → name *Arguments*
And expand *Arguments* into
the three alternatives.

To choose between expanding by 0, 1, or 2, a top-down parser must look beyond the name to see the next word.

$$\text{FIRST}^+(0) = \text{FIRST}^+(1) = \text{FIRST}^+(2) = \{ \text{name} \}$$

Left factoring productions 0, 1, & 2 fixes this problem.



Left Factoring Example

After left factoring, we have

0	<i>Factor</i>	$\rightarrow$	<u>name</u> <i>Args</i>
1	<i>Args</i>	$\rightarrow$	[<i>ArgList</i>]
2			(<i>ArgList</i>)
3			ϵ
4	<i>ArgList</i>		<i>Expr</i> <i>MoreArgs</i>
5	<i>MoreArgs</i>	$\rightarrow$	$_$ <i>Expr</i> <i>MoreArgs</i>
6			ϵ

Left factoring can transform some non-LL(1) grammars into LL(1) grammars.

There are languages for which no LL(1) grammar exists.

See exercise 3.12 in EaC2e.

Clearly, $\text{FIRST}^+(1)$ & $\text{FIRST}^+(2)$ are disjoint.

If neither (or [can follow *Factor*, then $\text{FIRST}^+(3)$ will be, too.

This tiny grammar now has the LL(1) property



FIRST and FOLLOW Sets

FIRST(α)

For some $\alpha \in (T \cup NT)^*$, define **FIRST(α)** as the set of tokens that appear as the first symbol in some string that derives from α

That is, $\underline{x} \in \text{FIRST}(\alpha)$ iff $\alpha \Rightarrow^* \underline{x}\gamma$, for some γ

FOLLOW(A)

For some $A \in NT$, define **FOLLOW(A)** as the set of symbols that can occur immediately after A in a valid sentential form

$\text{FOLLOW}(S) = \{\text{EOF}\}$, where S is the start symbol

To build **FOLLOW** sets, we need **FIRST** sets ...

Computing FIRST Sets



```
for each  $x \in T$ ,  $FIRST(x) \leftarrow \{x\}$ 
for each  $A \in NT$ ,  $FIRST(A) \leftarrow \emptyset$ 
while (FIRST sets are still changing) do
  for each  $p \in P$ , of the form  $A \rightarrow \beta$  do
    if  $\beta$  is  $B_1 B_2 \dots B_k$  then begin;
       $rhs \leftarrow FIRST(B_1) - \{\epsilon\}$ 
      for  $i \leftarrow 1$  to  $k-1$  by 1 while  $\epsilon \in FIRST(B_i)$  do
         $rhs \leftarrow rhs \cup (FIRST(B_{i+1}) - \{\epsilon\})$ 
      end // for loop
    end // if-then
    if  $i = k$  and  $\epsilon \in FIRST(B_k)$ 
      then  $rhs \leftarrow rhs \cup \{\epsilon\}$ 
     $FIRST(A) \leftarrow FIRST(A) \cup rhs$ 
  end // for loop
end // while loop
```

Outer loop is monotone increasing for FIRST sets

$\rightarrow |T \cup NT \cup \epsilon|$ is bounded, so it terminates

Inner loop is bounded by the length of the productions in the grammar

For *SheepNoise*:

$FIRST(Goal) = \{ \underline{b}aa \}$

$FIRST(SN) = \{ \underline{b}aa \}$

$FIRST(\underline{b}aa) = \{ \underline{b}aa \}$

Computing FIRST Sets



```
for each  $x \in T$ ,  $FIRST(x) \leftarrow \{x\}$ 
for each  $A \in NT$ ,  $FIRST(A) \leftarrow \emptyset$ 
while (FIRST sets are still changing) do
  for each  $p \in P$ , of the form  $A \rightarrow \beta$  do
    if  $\beta$  is  $B_1 B_2 \dots B_k$  then begin;
       $rhs \leftarrow FIRST(B_1) - \{\epsilon\}$ 
      for  $i \leftarrow 1$  to  $k-1$  by 1 while  $\epsilon \in FIRST(B_i)$  do
         $rhs \leftarrow rhs \cup (FIRST(B_{i+1}) - \{\epsilon\})$ 
      end // for loop
    end // if-then
    if  $i = k$  and  $\epsilon \in FIRST(B_k)$ 
      then  $rhs \leftarrow rhs \cup \{\epsilon\}$ 
     $FIRST(A) \leftarrow FIRST(A) \cup rhs$ 
  end // for loop
end // while loop
```

Outer loop is monotone increasing for FIRST sets

$\rightarrow |T \cup NT \cup \epsilon|$ is
For $SN \rightarrow \underline{baa}$ rates

Inner loop is bounded by the length of the productions in the grammar

For *SheepNoise*:

```
 $FIRST(Goal) = \{ \underline{baa} \}$ 
 $FIRST(SN) = \{ \underline{baa} \}$ 
 $FIRST(\underline{baa}) = \{ \underline{baa} \}$ 
```

Computing FIRST Sets



```
for each  $x \in T$ ,  $FIRST(x) \leftarrow \{x\}$ 
for each  $A \in NT$ ,  $FIRST(A) \leftarrow \emptyset$ 
while (FIRST sets are still changing) do
  for each  $p \in P$ , of the form  $A \rightarrow \beta$  do
    if  $\beta$  is  $B_1 B_2 \dots B_k$  then begin;
       $rhs \leftarrow FIRST(B_1) - \{\epsilon\}$ 
      for  $i \leftarrow 1$  to  $k-1$  by 1 while  $\epsilon \in FIRST(B_i)$  do
         $rhs \leftarrow rhs \cup (FIRST(B_{i+1}) - \{\epsilon\})$ 
      end // for loop
    end // if-then
    if  $i = k$  and  $\epsilon \in FIRST(B_k)$ 
      then  $rhs \leftarrow rhs \cup \{\epsilon\}$ 
     $FIRST(A) \leftarrow FIRST(A) \cup rhs$ 
  end // for loop
end // while loop
```

Outer loop is monotone increasing for FIRST sets

$\rightarrow |T \cup NT \cup \epsilon|$ is

For *Goal* $\rightarrow SN$ $\geq S$

Inner loop is bounded by the length of the productions in the grammar

For *SheepNoise*:

$FIRST(Goal) = \{ \underline{b}aa \}$

$FIRST(SN) = \{ \underline{b}aa \}$

$FIRST(\underline{b}aa) = \{ \underline{b}aa \}$

Computing FOLLOW Sets



```
for each  $A \in NT$ ,  $FOLLOW(A) \leftarrow \emptyset$ 
 $FOLLOW(S) \leftarrow \{EOF\}$ 
while ( $FOLLOW$  sets are still changing)
  for each  $p \in P$ , of the form  $A \rightarrow B_1 B_2 \dots B_k$ 
     $TRAILER \leftarrow FOLLOW(A)$ 
    for  $i \leftarrow k$  down to 1
      if  $B_i \in NT$  then // domain check
         $FOLLOW(B_i) \leftarrow FOLLOW(B_i) \cup TRAILER$ 
        if  $\epsilon \in FIRST(B_i)$  // add right context
          then  $TRAILER \leftarrow TRAILER \cup (FIRST(B_i) - \{\epsilon\})$ 
          else  $TRAILER \leftarrow FIRST(B_i)$  // no  $\epsilon \Rightarrow$  no right context
        else  $TRAILER \leftarrow \{B_i\}$  //  $B_i \in T \Rightarrow$  only 1 symbol
```

Classic Expression Grammar



0	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
1	<i>Expr</i>	$\rightarrow$	<i>Term Expr'</i>
2	<i>Expr'</i>	$\rightarrow$	$+$ <i>Term Expr'</i>
3		$ $	$-$ <i>Term Expr'</i>
4		$ $	ϵ
5	<i>Term</i>	$\rightarrow$	<i>Factor Term'</i>
6	<i>Term'</i>	$\rightarrow$	$*$ <i>Factor Term'</i>
7		$ $	$/$ <i>Factor Term'</i>
8		$ $	ϵ
9	<i>Factor</i>	$\rightarrow$	<u>number</u>
10		$ $	<u>id</u>
11		$ $	$($ <i>Expr</i> $)$

$\text{FIRST}^+(A \rightarrow \beta)$ is identical to $\text{FIRST}(\beta)$
except for productiond 4 and 8

$\text{FIRST}^+(\text{Expr}' \rightarrow \epsilon)$ is $\{\epsilon,), \text{eof}\}$

$\text{FIRST}^+(\text{Term}' \rightarrow \epsilon)$ is $\{\epsilon, +, -,), \text{eof}\}$

Symbol	FIRST	FOLLOW
<u>num</u>	<u>num</u>	$\emptyset$
<u>id</u>	<u>id</u>	$\emptyset$
$+$	$+$	$\emptyset$
$-$	$-$	$\emptyset$
$*$	$*$	$\emptyset$
$/$	$/$	$\emptyset$
$($	$($	$\emptyset$
$)$	$)$	$\emptyset$
<u>eof</u>	<u>eof</u>	$\emptyset$
ϵ	ϵ	$\emptyset$
<i>Goal</i>	$(, \text{id}, \text{num}$	eof
<i>Expr</i>	$(, \text{id}, \text{num}$	$), \text{eof}$
<i>Expr'</i>	$+, -, \epsilon$	$), \text{eof}$
<i>Term</i>	$(, \text{id}, \text{num}$	$+, -,), \text{eof}$
<i>Term'</i>	$*, / , \epsilon$	$+, -,), \text{eof}$
<i>Factor</i>	$(, \text{id}, \text{num}$	$+, -, *, / ,), \text{eof}$

Classic Expression Grammar



0	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
1	<i>Expr</i>	$\rightarrow$	<i>Term Expr'</i>
2	<i>Expr'</i>	$\rightarrow$	$+ \text{ Term Expr'}$
3		$ $	$- \text{ Term Expr'}$
4		$ $	ϵ
5	<i>Term</i>	$\rightarrow$	<i>Factor Term'</i>
6	<i>Term'</i>	$\rightarrow$	$* \text{ Factor Term'}$
7		$ $	$/ \text{ Factor Term'}$
8		$ $	ϵ
9	<i>Factor</i>	$\rightarrow$	<u>number</u>
10		$ $	<u>id</u>
11		$ $	(Expr)

Prod'n	FIRST+
0	$(, \text{id}, \text{num}$
1	$(, \text{id}, \text{num}$
2	$+$
3	$-$
4	$\epsilon,), \text{eof}$
5	$(, \text{id}, \text{num}$
6	$*$
7	$/$
8	$\epsilon, +, -,), \text{eof}$
9	<u>number</u>
10	<u>id</u>
11	$($

Building Top-down Parsers for LL(1) Grammars



Given an *LL(1)* grammar, and its FIRST & FOLLOW sets ...

- Emit a routine for each non-terminal
 - Nest of if-then-else statements to check alternate rhs's
 - Each returns true on success and throws an error on false
 - Simple, working (*perhaps ugly*) code
- Can automatically construct a recursive-descent parser

Improving matters

- Nest of if-then-else statements may be slow
 - Good case statement implementation would be better
- What about a table to encode the options?
 - Interpret the table with a skeleton, as we did in scanning

I don't know of a system
that does this ...

Building Top-down Parsers



Strategy

- Encode knowledge in a table
- Use a standard "skeleton" parser to interpret the table

Example

- The non-terminal *Factor* has 3 expansions
 - (*Expr*) or Identifier or Number
- Table might look like:

0	Goal	→	Expr
1	Expr	→	Term Expr'
2	Expr'	→	+ Term Expr'
3			- Term Expr'
4			ε
5	Term	→	Factor Term'
6	Term'	→	* Factor Term'
7			/ Factor Term'
8			ε
9	Factor	→	<u>number</u>
10			<u>id</u>
11			(Expr)

		Terminal Symbols								
		+	-	*	/	Id.	Num.	(	)	EOF
Non-terminal Symbols	<u>Factor</u>	—	—	—	—	10	9	11	—	—

Cannot expand *Factor* into an operator ⇒ error

Expand *Factor* by rule 9 with input "number"

Building Top-down Parsers



Building the complete table

- Need a row for every NT & a column for every T

LL(1) Expression Parsing Table



	+	-	*	/	Id	Num	(	)	EOF
Goal	—	—	—	—	0	0	0	—	—
Expr	—	—	—	—	1	1	1	—	—
Expr'	2	3	—	—	—	—	—	4	4
Term	—	—	—	—	5	5	5	—	—
Term'	8	8	6	7	—	—	—	8	8
Factor	—	—	—	—	10	9	11	—	—

Row we built earlier

Table differs from EaC2e because the order of productions in *Factor* differs.

Building Top-down Parsers



Building the complete table

- Need a row for every NT & a column for every T
- Need an interpreter for the table (*skeleton parser*)



LL(1) Skeleton Parser

```
word ← NextWord()           // Initial conditions, including
push EOF onto Stack         // a stack to track local goals
push the start symbol,  $S$ , onto Stack
TOS ← top of Stack

loop forever
  if TOS = EOF and word = EOF then
    break & report success // exit on success
  else if TOS is a terminal then
    if TOS matches word then
      pop Stack             // recognized TOS
      word ← NextWord()
    else report error looking for TOS // error exit
  else                     // TOS is a non-terminal
    if TABLE[TOS,word] is  $A \rightarrow B_1 B_2 \dots B_k$  then
      pop Stack             // get rid of  $A$ 
      push  $B_k, B_{k-1}, \dots, B_1$  // in that order
    else break & report error expanding TOS
  TOS ← top of Stack
```



Building Top-down Parsers

Building the complete table

- Need a row for every NT & a column for every T
- Need a table-driven interpreter for the table
- Need an algorithm to build the table

Filling in $TABLE[X,y]$, $X \in NT$, $y \in T$

1. entry is the rule $X \rightarrow \beta$, if $y \in FIRST^+(X \rightarrow \beta)$
2. entry is *error* if rule 1 does not define

If any entry has more than one rule, G is not $LL(1)$

We call this algorithm the $LL(1)$ table construction algorithm



Recap of Top-down Parsing

- Top-down parsers build syntax tree from root to leaves
- Left-recursion causes non-termination in top-down parsers
 - Transformation to eliminate left recursion
 - Transformation to eliminate common prefixes in right recursion
- FIRST, FIRST⁺, & FOLLOW sets + LL(1) condition
 - LL(1) uses left-to-right scan of the input, leftmost derivation of the sentence, and 1 word lookahead
 - LL(1) condition means grammar works for predictive parsing
- Given an LL(1) grammar, we can
 - Build a recursive descent parser
 - Build a table-driven LL(1) parser
- LL(1) parser doesn't build the parse tree
 - Keeps lower fringe of partially complete tree on the stack

Grammar and Sets for the LL(1) Construction



0	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
1	<i>Expr</i>	$\rightarrow$	<i>Term Expr'</i>
2	<i>Expr'</i>	$\rightarrow$	$+ \text{ Term Expr'}$
3		$ $	$- \text{ Term Expr'}$
4		$ $	ϵ
5	<i>Term</i>	$\rightarrow$	<i>Factor Term'</i>
6	<i>Term'</i>	$\rightarrow$	$* \text{ Factor Term'}$
7		$ $	$/ \text{ Factor Term'}$
8		$ $	ϵ
9	<i>Factor</i>	$\rightarrow$	<u>number</u>
10		$ $	<u>id</u>
11		$ $	(Expr)

Right-recursive variant of the classic expression grammar

Comp 412, Fall 2012

Prod'n	FIRST+
0	(, <u>id</u> , <u>num</u>
1	(, <u>id</u> , <u>num</u>
2	+
3	-
4	ϵ ,), eof
5	(, <u>id</u> , <u>num</u>
6	*
7	/
8	ϵ , +, -,), eof
9	<u>number</u>
10	<u>id</u>
11	(

Augmented FIRST sets for the grammar